SECURE AND RESILIENT VIRTUALIZED DATA CENTRE DESIGN

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102241	**A.VISHNU VARDHAN**
111624102242	**VISHWAJITH R**
111624102247	**YUVARAJ U**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **A.VISHNU VARDHAN, VISHWAJITH R and YUVARAJ U**, bearing **Register Numbers 111624102241, 111624102242 and 111624102247** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"Secure and Resilient Virtualized Data Centre Design"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student	**Project Guide**
A.VISHNU VARDHAN	Dr. J GNANA JESLIN
VISHWAJITH R	
YUVARAJ U	

ABSTRACT

A virtualized data centre runs many services on a small number of shared hosts, networks, power feeds and storage arrays. A design is judged on three questions that depend on one another: does the capacity fit and survive the loss of a host, are the network zones separated so that an attacker cannot reach the databases from the internet, and does any single failure take a service down. Design reviews usually answer them by checklist. This mini project builds a tool that answers them by calculation, using only the Python standard library. A design is described as hosts, racks, power feeds, switches, cooling units, firewalls, storage arrays, virtual machines and firewall rules. The capacity module places VMs with anti-affinity and tests N+1. The security module applies first-match firewall semantics, compares the rules with an intended three-tier policy, finds routes from the internet to the critical zones and scores a hardening checklist. The resilience module fails every component and every pair of components in turn, finds single points of failure, simulates the availability of each service, and gives a simplified tier classification. Three reference designs were evaluated. The basic flat design scored 45 out of 100, with 2 critical findings, a one-hop route from the internet to the database, 6 single points of failure and 4.6 hours of simulated downtime a year; the resilient design scored 98, with no route shorter than three hops, no single point of failure and 2.6 minutes of downtime a year. An automatic remediation lifted the basic design to 87, but left nine process items for people. Every one of 476 injected firewall mistakes was detected. A command-line tool and a browser interface are provided, and 34 automated tests pass.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	3
3	Tools and Technologies	5
4	Architecture Diagram	6
5	Methodology	8
6	Results	12
7	Conclusion	21
8	Reference	23

1. INTRODUCTION

1.1 Background

A data centre is the building, power, cooling and network in which an organisation's computers run. Virtualization changed how it is used: instead of one application on one server, a hypervisor lets many virtual machines (VMs) share a physical host, so a few dozen hosts can carry hundreds of services [1][3]. This sharing is what makes a modern data centre efficient, and also what makes it fragile. A power feed, a switch or a cooling unit now serves many hosts and therefore many services, and a misconfigured virtual network can expose every VM at once. The design of the data centre must therefore be done with three concerns in mind at the same time: capacity, security and resilience.

Capacity asks whether the VMs fit on the hosts, and whether they would still fit if a host failed (the N+1 rule). Replicas of one service must not share a host, otherwise the loss of the host would take both down; assigning VMs to hosts under such constraints is a bin-packing problem, which is hard in general and is solved by heuristics [5]. **Security** asks whether the network is divided into zones, whether the firewall allows only the flows the design needs, and whether the hypervisor and its management are hardened; guidance for this is given by NIST for virtualization [7] and for firewall policy [8], and rests on the old principle of least privilege [6]. **Resilience** asks what happens when a component fails. The Uptime Institute's tier classification [2] describes how redundancy of power, cooling and network paths raises the expected availability from about 99.67% for a basic site to about 99.98% for a concurrently maintainable one, and the main task of a designer is to find and remove the single points of failure.

Table 1.1: The three concerns, what is checked and by which method in this project

Concern	Question	What is checked	Method in the tool
Capacity	Do the VMs fit, and still fit after a failure?	CPU and memory per host, replicas on different hosts and racks, loss of any one host	placement heuristics, N+1 test
Security	Can an attacker reach the critical zones?	firewall rules against the intended policy, routes from the internet, hardening checklist	first-match rule evaluation, graph search, scoring
Resilience	Which failure stops a service?	loss of any host, rack, power feed, switch, cooling unit, firewall or storage array, and of any two	failure scenarios, restart with the remaining capacity, simulation

1.2 Problem Statement

Design reviews of virtualized data centres are usually done on paper, with a checklist and a diagram. The checklist says that the network should be segmented and the power should be

redundant; it does not say whether *this* design is, and it cannot find the cases where two sound decisions combine badly: for example a pair of database VMs placed in the same rack, so that the loss of the rack defeats the replication, or a firewall whose second rule is unreachable because the first rule already matches. Such faults are found, if at all, after an incident or a security audit.

The problem addressed here is to build a tool that takes a precise description of a virtualized data centre design and (i) checks the capacity, including N+1; (ii) evaluates the firewall rules and the routes from the internet to the critical zones; (iii) finds the single points of failure by failing every component, and every pair of components, in turn; (iv) estimates the availability of each service; and (v) applies the fixes that can be applied automatically, and says what remains for people to do.

1.3 Motivation

The first motivation is that the mistakes are cheap to make and expensive to find. A flat network with an allow-all rule is the default of many small installations, and in the reference design of this project it gives a route of a single hop from the internet to the database (Section 6.2). The second is that redundancy is easy to claim and hard to verify: a design with two power feeds is not resilient if the two feeds meet in one distribution board, or if the two replicas of a service share a rack. The third is educational. The subject of the course covers virtualization, data-centre architecture, security and resilience as separate units, and a design tool that evaluates one design on all of them at once shows how the units interact.

1.4 Scope

The tool evaluates a *model* of a data centre, not a real one. A design consists of hosts with CPU and memory, racks, power feeds, switches, cooling units, firewalls, storage arrays, VMs with their tier and replica group, and an ordered list of firewall rules between six zones (internet, dmz, app, db, mgmt, storage). It covers a three-tier service with a management zone. It does not scan a real network, does not read the configuration of a real hypervisor, does not model software vulnerabilities, denial-of-service attacks or insider threats, and uses assumed failure rates (Table 5.2). A high score therefore means that the *design* is sound under these assumptions, not that a real installation is safe. The limits are listed in Section 7.1.

2. OBJECTIVES

The main objective is to design and implement an evaluator for the capacity, security and resilience of a virtualized data centre design, and to show with experiments that it finds real weaknesses and that its automatic fixes improve a weak design. The specific objectives are:

- To model a data centre as hosts, racks, power feeds, switches, cooling units, firewalls, storage arrays, VMs and firewall rules.
- To place VMs on hosts under CPU, memory and anti-affinity constraints with three heuristics, and to test whether the cluster survives the loss of any host.
- To evaluate firewall rules with first-match semantics, compare them with the intended policy, and search the zone graph for routes from the internet to the critical zones.
- To score a hardening checklist for the hypervisor, management, storage and backups.
- To fail every component, and every pair of components, find the single points of failure, and simulate the availability of each service including shared-component failures.
- To provide reference designs (basic, segmented, resilient) and an automatic remediation, and to measure what it improves.
- To offer a command-line interface and a browser interface with an editable firewall.

Table 2.1: Functional requirements

ID	Requirement
FR1	Describe a design as hosts, racks, feeds, switches, cooling, firewalls, arrays, VMs and ordered firewall rules
FR2	Place VMs with first fit, best fit and first-fit-decreasing under CPU, memory and anti-affinity constraints
FR3	Report utilisation and test N+1 (survival of every single host failure)
FR4	Evaluate the rules with first-match semantics and report findings with severity and a fix
FR5	Find the routes from the internet to the db, mgmt and storage zones and their length
FR6	Fail every component and every pair, restart VMs on the remaining capacity, and list the services that stop
FR7	Simulate the availability of every tier and of the service end to end
FR8	Apply the technical fixes automatically and list what is left for people

Table 2.2: Non-functional requirements

Requirement	How it is met
Correctness	34 automated tests, including hand-checked failure scenarios, injected firewall faults and simulation against formulas
Repeatability	Reference designs are generated deterministically; simulations use a seed
Transparency	Every finding names the rule and the reason; every score is a documented formula
Portability	Python standard library only
Speed	A design of 320 VMs is evaluated in under a second

3. TOOLS AND TECHNOLOGIES

The tool is written in Python and uses only the standard library, so that each rule of evaluation can be read next to its description. Table 3.1 lists what was used.

Table 3.1: Tools and technologies used

Item	Tool	Purpose
Language	Python 3.10 or newer (developed and tested on 3.13)	Model, analysis, evaluator, interfaces
Libraries	Standard library only: `dataclasses`, `random`, `itertools`, `copy`, `http.server`, `json`, `unittest`	Design model, failure simulation, server, tests
Browser interface	One HTML file with CSS and plain JavaScript	Five tabs: overview, security, resilience, capacity, remediation
Server	`http.server.ThreadingHTTPServer`	JSON endpoints used by the page
Testing	`unittest`	34 automated tests
Charts for this report	matplotlib	Figures 6.1 to 6.5 (not needed to run the tool)
Editor and shell	Visual Studio Code, PowerShell or any terminal	Development

3.1 Running the project

The project is a folder named `dcdesign`. No installation is needed. From inside the folder:

```
python -m unittest discover -s tests          # the 34 automated tests (a few
seconds)
python -m dcd.cli evaluate basic              # evaluate a reference design
python -m dcd.cli harden basic                # apply the automatic fixes and
show before and after
python -m dcd.cli compare --vms 80            # basic, segmented and resilient
side by side
python -m dcd.server 8000                     # browser interface at
http://127.0.0.1:8000
python bench.py                               # the experiments of Chapter 6
```

4. ARCHITECTURE DIAGRAM

The tool has four layers (Figure 4.1). The presentation layer is the command line, the browser page and the HTTP server behind the page. The evaluator combines the three analyses into one result, builds the reference designs for any number of VMs, and applies the automatic remediation. The analysis layer has one module for each concern: capacity, security and resilience. At the bottom, the model describes a design with a few dataclasses and holds the constants shared by the analyses, such as the intended policy and the hardening checklist. A layer calls only the layer below it, so each analysis can be tested without an interface.

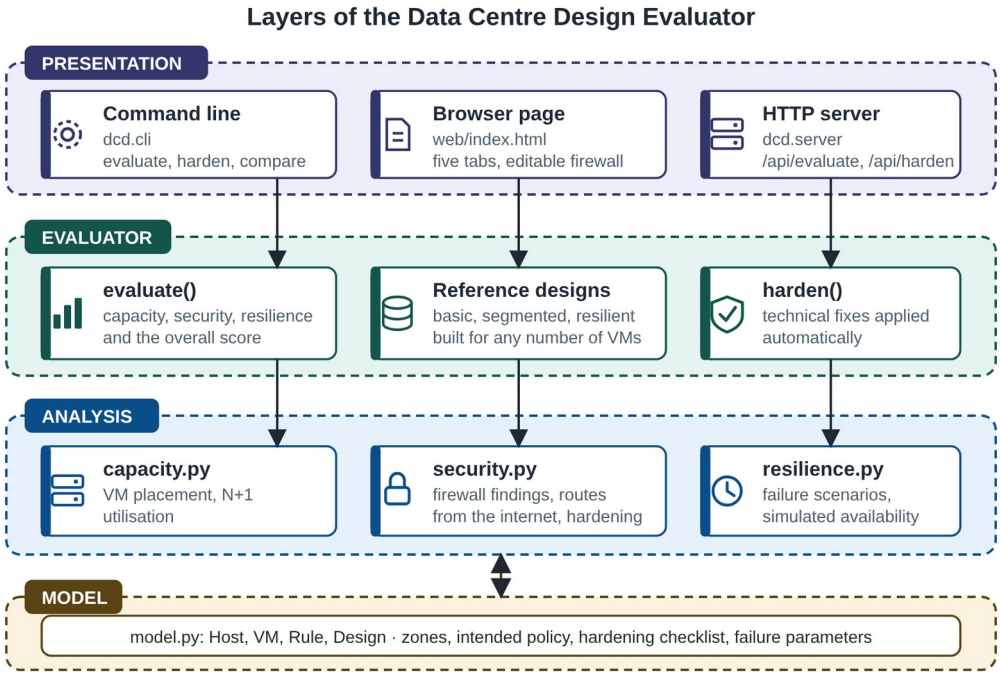


Figure 4.1: Layers of the Data Centre Design Evaluator

Table 4.1: Module responsibilities

File	Lines	Responsibility
`dcd/model.py`	66	Host, VM, Rule and Design classes; zones; hardening checklist
`dcd/capacity.py`	75	VM placement heuristics, utilisation, N+1 test
`dcd/security.py`	102	Firewall evaluation, findings, routes from the internet, hardening score
`dcd/resilience.py`	149	Failure scenarios, single points of failure, availability simulation, tier reading
`dcd/designs.py`	103	Reference designs and automatic remediation
`dcd/evaluate.py`	18	Combined evaluation and overall score
`dcd/cli.py`, `dcd/server.py`	119	Command line and HTTP server
`web/index.html`	110	Browser page
`tests/test_dcd.py`	266	34 automated tests
`bench.py`	84	Experiments of Chapter 6

4.1 The reference designs

Three reference designs are generated for any number of VMs, so that the tool can be tried without writing a design by hand. All three run the same service: web VMs in the dmz zone, application VMs in the app zone, a pair of database VMs in the db zone and a pair of management VMs in the mgmt zone, with replicas grouped in pairs. The **basic** design is a flat network with one allow-all rule and a default of allow, one rack, one power feed, one switch, one cooling unit, one firewall and one unmirrored storage array. The **segmented** design has the same hardware but separates the six zones and allows only the intended flows with a default of deny. The **resilient** design (Figure 4.2) adds a second of every path, two racks, mirrored arrays and a spare host. Figure 4.2 shows the resilient design: users reach the dmz through a firewall pair; the app and db zones can be reached only through the chain of intended flows; the management zone is reached from nowhere on the internet; and every host sees both power feeds, both switches and both cooling units.

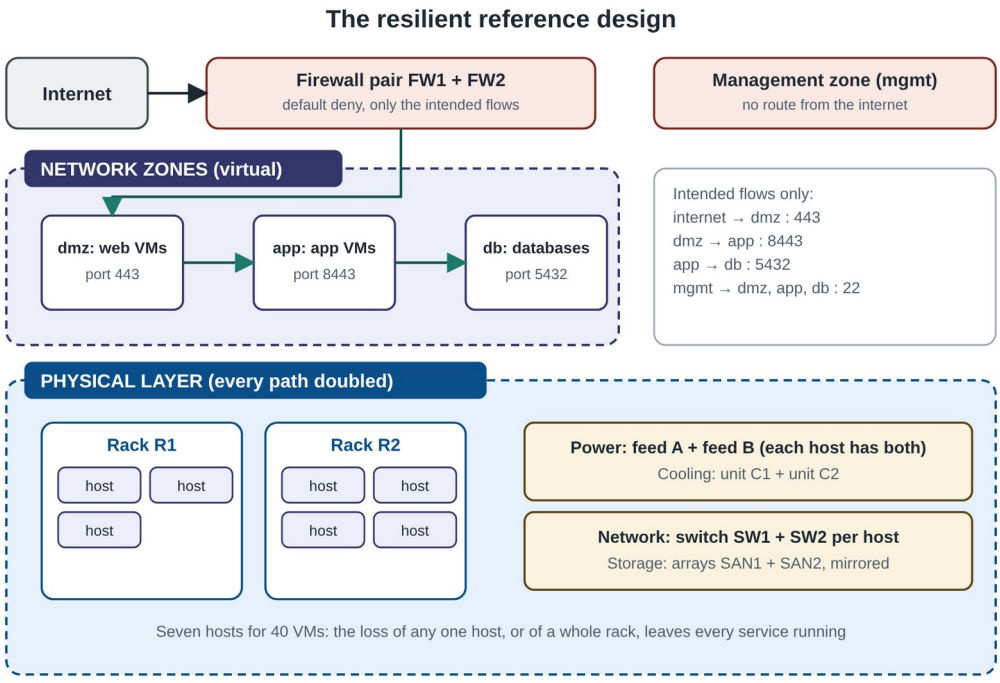


Figure 4.2: The resilient reference design

5. METHODOLOGY

5.1 Overall approach

A design is evaluated in three independent analyses whose results are combined into an overall score. Each analysis is deliberately simple, so that a student can follow a result back to its cause: a score falls because of a named finding, a scenario fails because of a named component. The experiments of Chapter 6 then use the evaluator on the reference designs, on injected mistakes and on random workloads, and measure what the automatic remediation changes.

5.2 Capacity: placement and N+1

Every VM has a CPU and memory demand and a replica group; every host has a CPU and memory capacity, a rack and a set of paths. Placement assigns VMs to hosts so that no host is over-committed and no two VMs of one group share a host. Among the hosts that fit, the rule prefers a rack that does not yet hold a member of the group, so that replicas are also spread over racks. Three heuristics are implemented [5]: **first fit** takes the first host with room; **best fit** takes the host that would be left with the least space; **first fit decreasing** sorts the VMs by size, largest first, and then applies first fit. The lower bound on the number of hosts is the larger of total CPU and total memory divided by the capacity of one host.

A cluster is **N+1** when every single host can fail and its VMs can be restarted on the free capacity of the other hosts, again respecting the replica rule. The test removes each host in turn, keeps the other VMs where they are, and tries to place the displaced VMs. The same restart step is used by the failure scenarios of Section 5.4.

5.3 Security: firewall rules, routes and hardening

The firewall is a list of rules evaluated in order, each with a source zone, a destination zone, a port, an action and a flag for encryption. As in a real firewall the **first matching rule decides**, and a flow that no rule matches gets the default action. The intended policy of a three-tier service is small: internet to dmz on 443, dmz to app on 8443, app to db on 5432, and management to the three tiers on 22 (Figure 5.1). Anything else is a deviation.

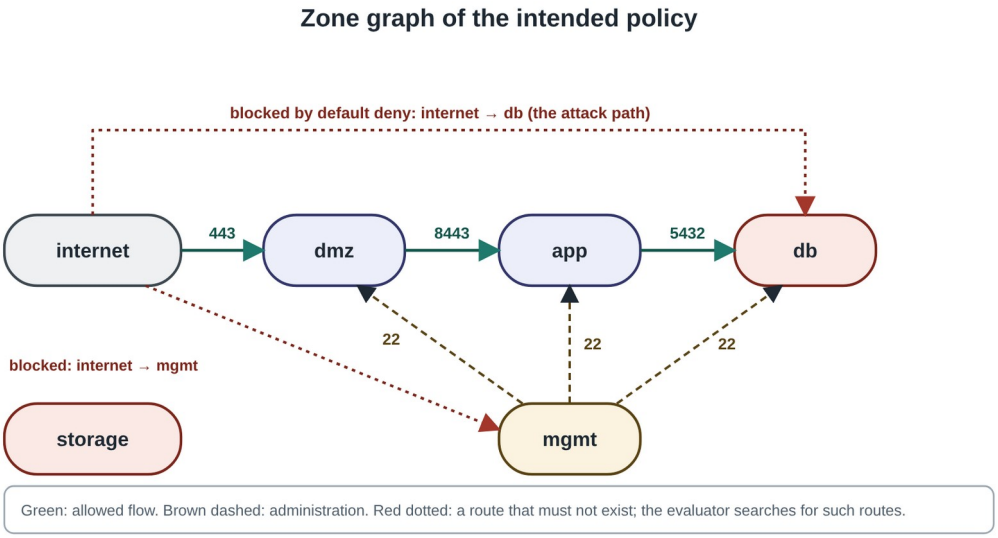


Figure 5.1: Zone graph of the intended policy

Table 5.1: Findings of the firewall evaluation

Finding	Severity	Condition	Penalty
default-allow	critical	the default action is allow	25
any-any	critical	an allow rule with a wildcard in both source and destination	25
unauthorised-flow	critical, high or medium	an allow rule that is not in the intended policy: critical if it leads from the internet or dmz to db, mgmt or storage, high if it leads to those zones from elsewhere, medium otherwise	25 / 12 / 6
broad-rule	high	an allow rule with a wildcard in the source, destination or port	12
unencrypted	medium	a rule from the internet, or to db, mgmt or storage, without encryption	6
shadowed-rule	low	a rule that can never match because an earlier rule already covers it [9]	2
missing-flow	low	an intended flow that the rules block	2

The **policy score** is 100 minus the sum of the penalties (not below 0). A second source of penalty is the **reachability** of the critical zones. The evaluator builds a graph whose nodes are the zones and whose edges are the flows allowed by the rules, and searches it for simple paths of up to four hops from the internet to the db, mgmt and storage zones, in the manner of attack-graph analysis [10]. A route of a single hop to any critical zone adds 20 to the penalty, and a two-hop route to mgmt or storage adds 10, because such a route does not need any step in the middle that an attacker would have to compromise. The **hardening score** is the weighted share of the items of an 11-item checklist that are in place (patched hypervisor, multi-factor administration, disk encryption, immutable backups, central logging, secure boot, least-privilege roles, VLAN isolation, intrusion detection, time synchronisation and audit, snapshot policy) in the manner of the NIST guidance [7]. The **security score** is 0.6 × policy score + 0.4 × hardening score.

5.4 Resilience: failure scenarios, availability and tier

The components of a design are its hosts, racks, power feeds, switches, cooling units, firewalls and storage arrays. A host is unavailable if the host or its rack fails, **or if all** of its power feeds, **or all** of its switches, **or all** of its cooling units, **or all** of the firewalls fail: redundant paths protect it until the last one is gone. In a **scenario** a set of components fails. The VMs of the unavailable hosts are restarted on the free capacity of the other hosts; VMs whose only copy of the data was on a failed unmirrored array stay down. A tier (web, app, db, mgmt) is a service and is **down** when no VM of it is running. A scenario is **tolerated** if no service is down and no critical VM is lost; it is marked *degraded* if some non-critical VM could not be restarted. A **single point of failure** is a component whose failure alone is not tolerated. The **resilience score** is the percentage of single-component scenarios that are tolerated.

The scenarios show what *can* happen, not how often. For that the evaluator simulates the availability of each service. Every component alternates between working and failing with exponentially distributed times of a given mean (Table 5.2); the down intervals of a VM are the union of the intervals of its host, rack and array and the *intersection* of the intervals of its parallel paths (both feeds, both switches, both cooling units, both firewalls); a pair of replicas is down when both are down, and the whole service when its web, app or db tier is down. Because replicas that share a rack or a feed fail together, the simulation reproduces correlated failures that a formula using independent copies would miss; a test confirms that two replicas in one rack are far less available than the independent formula says.

Table 5.2: Failure parameters used by the simulation (model assumptions)

Component	Mean time between failures	Mean time to repair
Host	2,000 h	4 h
Rack (its distribution unit)	500,000 h	8 h
Switch	100,000 h	4 h
Power feed	17,520 h	4 h
Cooling unit	50,000 h	8 h
Firewall	100,000 h	4 h
Storage array	200,000 h	8 h

The values are plausible orders of magnitude, not vendor figures. VMs are not restarted automatically in the simulation, which is conservative: a failed host keeps its VMs down for the repair time. The tier reading is a simplified version of the idea of the Uptime Institute [2]: a design with hosts at N+1, dual power, dual network and firewalls, redundant cooling, mirrored storage and no single point of failure is called **Tier III-like**; one with

N+1 hosts and some redundant power or cooling **Tier II-like**; anything else **Tier I-like**. The top tier is never awarded because it depends on the physical separation of the paths, which the model cannot see.

5.5 Automatic remediation

`harden()` applies the changes that are safe to make by a program, to a copy of the design: it sets the default action to deny; removes every allow rule that has a wildcard or is not in the intended policy; adds any intended flow that is missing; requires encryption on every rule; adds a second firewall, a second power feed, a second switch and a second cooling unit where there is only one; spreads the hosts over two racks; mirrors the storage over two arrays; and adds hosts until the cluster is N+1 and also survives the loss of a whole rack. It does *not* change the hardening checklist: patching, multi-factor authentication, backups and the like need people and process, so they are returned as a list of what is left to do. Applying the function to its own output changes nothing, which a test checks.

5.6 Scores and testing

The overall score is 0.25 × capacity + 0.40 × security + 0.35 × resilience, where the capacity score is 100 when the cluster is N+1 and falls with the share of hosts whose loss cannot be absorbed. The weights are a judgement of the authors and can be changed. The 34 tests cover the placement rules (capacity never exceeded, replicas apart, racks preferred), first-match semantics, each kind of firewall finding on an injected fault, the route search, the failure scenarios on small hand-built designs (single feed, single firewall, unmirrored array, two simultaneous failures), the simulation against the series and parallel formulas and against the effect of a shared rack, the reference designs and the remediation, and the command line and HTTP interfaces.

6. RESULTS

6.1 Test results

All 34 automated tests pass (`python -m unittest discover -s tests`, run time a few seconds). Table 6.1 lists them by group.

Table 6.1: Automated tests

Group	Tests	What is checked
Placement and N+1	7	capacity never exceeded on 60 random workloads; first fit decreasing on a bad order; anti-affinity; racks preferred; too-big VM; lower bound; N+1 with two and three hosts
Firewall	7	first match and default action; the intended policy is clean; six injected fault kinds detected; blocked intended flow; internet to database is critical; scores ordered; hardening score
Failure scenarios	7	dual feed survives; single feed is a single point of failure; single firewall; unmirrored array; two failures defeat a dual path; restart uses free capacity; component list
Availability simulation	4	single VM against the series formula; two replicas against the parallel formula; shared rack worse than the formula; same seed same result
Designs and remediation	7	reference designs valid; scores ordered; resilient has no single point of failure; basic has the six expected ones; remediation improves everything technical; idempotent; original untouched
Interfaces	2	command line; HTTP endpoints, an allow-all rule set, the remediation and the 400 reply for a bad rule

6.2 Four designs compared

Table 6.2 and Figure 6.1 give the evaluation of the three reference designs for 40 VMs, and of the basic design after the automatic remediation. The basic design is dangerous in both respects. Its firewall is a single allow-all rule with a default of allow, so there are 2 critical findings, the internet can reach the database, the management zone and the storage zone in one hop, and there are 41 different routes of up to four hops to the database; the security score is 7.2. Its hardware has six single points of failure: array SAN1, cooling C1, edge FW1, feed A, rack R1, switch SW1. Each of them stops the service when it fails, so only half of the 12 single-failure scenarios are survived. The segmented design removes the security problems (score 82.4, one route to the database of three hops through the dmz and app zones, none to mgmt and storage), but has the same hardware and so the same six single points of failure. The resilient design has none: all 19 scenarios are tolerated, of which 2 (the loss of a whole rack) leave some VMs down. It needs only one host more than the basic design (7 against 6).

Table 6.2: Four designs for 40 VMs

Design	Hosts	CPU used	Security score	Critical findings	Internet to database	Single points of failure	Resilience score	Overall
Basic	6	69%	7.2	2	1 hop(s), 41 route(s)	6	50	45.4
Segmented	6	69%	82.4	0	3 hop(s), 1 route(s)	6	50	75.5
Resilient	7	59%	94.4	0	3 hop(s), 1 route(s)	0	100	97.8
Basic after fixes	7	59%	67.2	0	3 hop(s), 1 route(s)	0	100	86.9

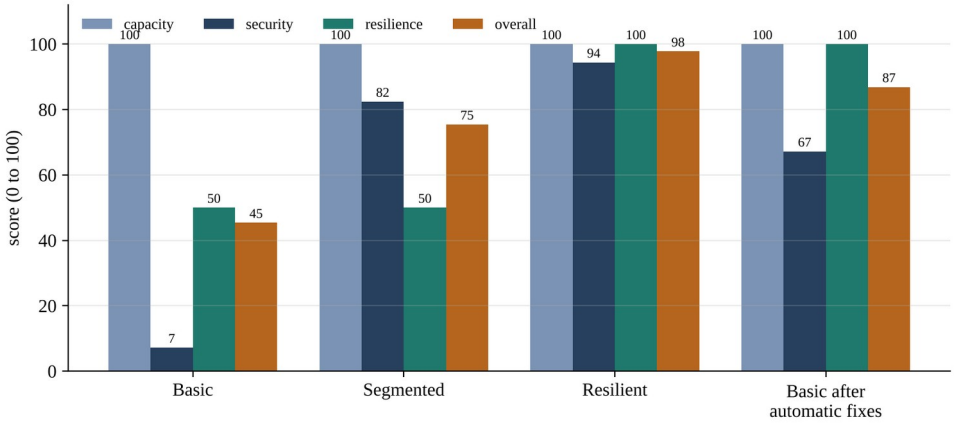


Figure 6.1: Capacity, security, resilience and overall score of the four designs

6.3 Simulated availability

Figure 6.2 shows the simulated downtime of the whole service (5,000 years, web, app and db all working). The basic and segmented designs are down 276 minutes a year (4.6 hours, 99.948% available); segmentation improves security and has no effect on availability, as it should. The resilient design is down 2.6 minutes a year, a reduction of 99%. The remaining downtime comes from the database pair, the two VMs that cannot be spread over more than two hosts: both of them have to fail for the database to stop, which happens a few minutes a year. For comparison, the Uptime Institute [2] describes about 99.67% for a Tier I site, 99.74% for Tier II and 99.98% for Tier III; the evaluated designs come out better than these figures because they model only the failures of Table 5.2 and not, for example, maintenance windows, software faults or human error [11][13], and the simulation should be read for the ratio between designs and not as a forecast.

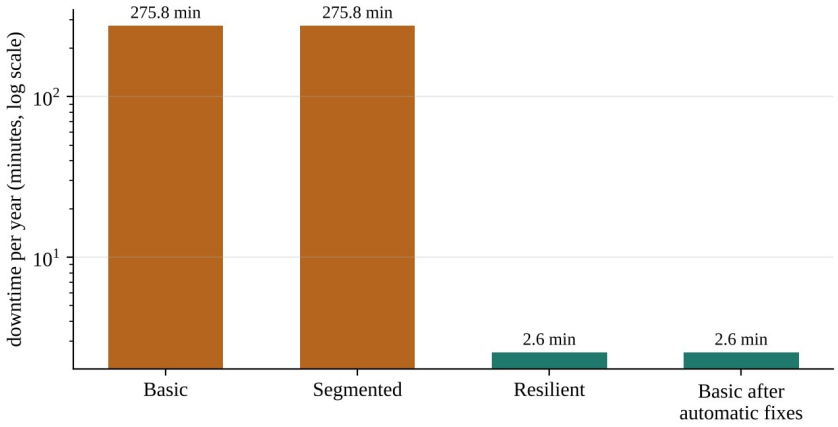


Figure 6.2: Simulated downtime of the whole service (log scale)

6.4 Failures of one and of two components

A design that survives every single failure can still be defeated by two. Failing every pair of the 19 components of the resilient design (171 pairs), 162 pairs are tolerated (94.7%). The 9 pairs that are not tolerated are exactly the ones that a designer would expect: the two cooling units, the two firewalls, the two power feeds and the two switches (the four redundant pairs), the two racks, and four combinations of a host with the rack of another host. The last group is instructive: the loss of host h2 and of rack R1 together takes down both database VMs, and the cluster has room to restart only one of them. The service survives, but a critical VM is lost, and such a scenario is counted as not tolerated. It shows that N+1 protects against one failure, not two, and that the choice between N+1 and N+2 is a decision about the cost of the next failure. The automatically remediated design has the same figure.

6.5 Automatic remediation

Applying `harden()` to the basic design makes 15 changes (Table 6.3), and the overall score rises from 45.4 to 86.9 (Figure 6.3). The firewall policy score goes from 0 to 100: the allow-all rule is replaced by the six intended flows and the default becomes deny. The resilience score goes from 50 to 100 and the six single points of failure disappear; the design needs 1 extra host for this (7 against 6), and its simulated downtime falls to 2.6 minutes a year. The security score, however, reaches only 67.2, because the hardening checklist stays at 18%: the tool cannot patch the hypervisor, enrol administrators in multi-factor authentication or set up immutable backups. These nine items are returned as the list of what remains for people. This is the honest outcome of the experiment: configuration problems can be repaired by a program, process problems cannot, and a tool that claims to fix both is misleading.

Table 6.3: Changes made by the automatic remediation of the basic design

#	Change
1	default action set to deny
2	removed rule allow any -> any port any
3	added intended flow app -> db port 5432
4	added intended flow dmz -> app port 8443
5	added intended flow internet -> dmz port 443
6	added intended flow mgmt -> app port 22
7	added intended flow mgmt -> db port 22
8	added intended flow mgmt -> dmz port 22
9	second firewall added
10	second power feed to every host
11	second switch to every host
12	second cooling unit for every row
13	hosts spread over two racks
14	storage mirrored over two arrays
15	1 extra host(s) added so that the loss of a host or of a whole rack can be absorbed

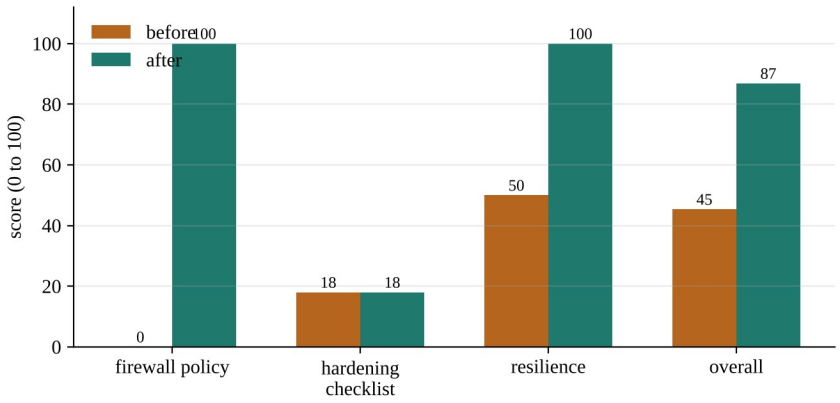


Figure 6.3: Scores of the basic design before and after the automatic remediation

6.6 Detection of firewall mistakes

To test the firewall evaluation, 493 random allow rules (random source and destination zone, and one of the ports 443, 22, 5432, 8443, 3306 or a wildcard) were added one at a time to the intended policy. Of the 476 rules that are not part of the intended policy, all 476 were flagged (406 as unauthorised flows, 70 as broad rules). The 17 rules that were copies of an intended flow were never reported as violations; they appear only as shadowed (low) at most. Both numbers are properties of the rule evaluation and were expected; they show that it behaves consistently and that the intended policy is the reference. They do not show that the policy itself is complete, which is a design judgement. Six hand-made faults (allow-all, internet to the database, a wildcard rule, a default of allow, an unencrypted flow and a shadowed rule) are each detected by a test.

6.7 Placement heuristics

Figure 6.4 compares the three placement heuristics on 300 random workloads of 40 VMs with replica pairs and eight VM shapes, on hosts of 32 CPUs and 128 GB. First fit used 7.85 hosts on average, best fit 7.83 and first fit decreasing 8.05. The theoretical minimum was reached in 67%, 68% and 46% of the runs. The result is not what is usually taught: sorting the VMs did not help here. The reason is that placement is two-dimensional (CPU and memory) and constrained (replicas on separate hosts), and a sort by the sum of the normalised sizes does not match either constraint well; in this workload the differences are small, 0.2 hosts out of 8, but they show that a textbook rule should be measured before it is trusted. The reference designs use first fit decreasing, and the difference of one host in a few runs does not change their conclusions.

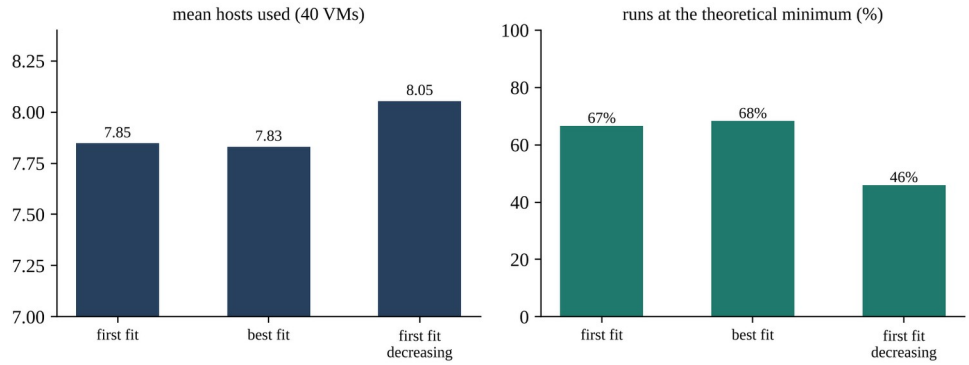


Figure 6.4: Hosts needed by the three placement heuristics

6.8 Scalability

Figure 6.5 shows the time to evaluate the resilient design for 20 to 320 VMs: 0.04 seconds for 20 VMs (5 hosts) and 0.75 seconds for 320 VMs (34 hosts), including 500 simulated years. The time grows a little faster than linearly because every scenario re-places the displaced VMs and the number of scenarios grows with the number of hosts. The evaluator is therefore fast enough for interactive use on designs of a few hundred VMs.

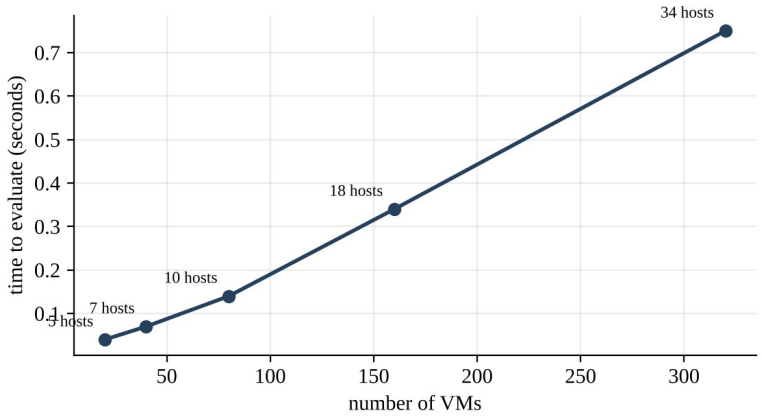


Figure 6.5: Time to evaluate a design against its size

6.9 Browser interface

Figures 6.6 to 6.9 show the browser interface (`python -m dcd.server`). The selector at the top chooses a reference design and the number of VMs.

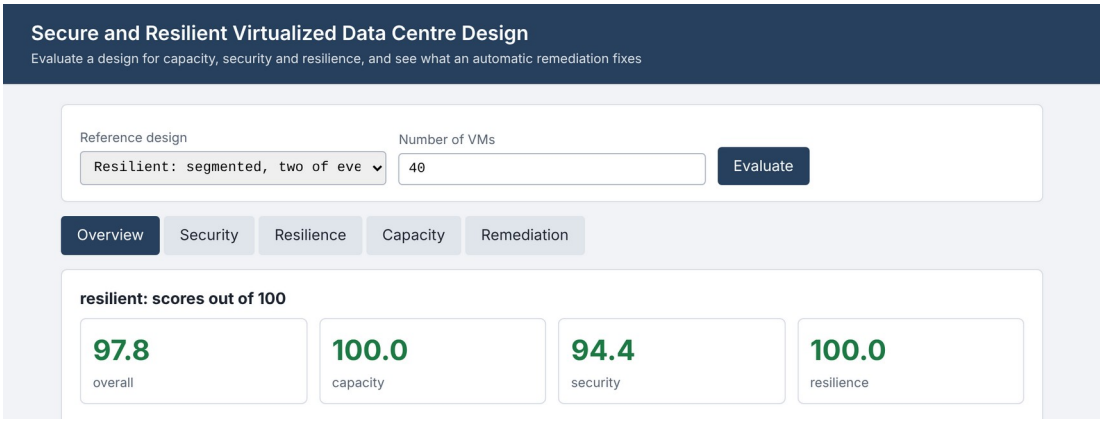


Figure 6.6: Overview tab: scores of the resilient design

In Figure 6.6 the resilient design has an overall score of 97.8: capacity 100, security 94.4 and resilience 100. The table beneath the cards gives the hosts and the survival of a host failure, the number of findings, the single points of failure and the simulated availability of the whole service (the figure on the screen uses 1,000 simulated years, so it differs a little from the 5,000-year value of Table 6.2).

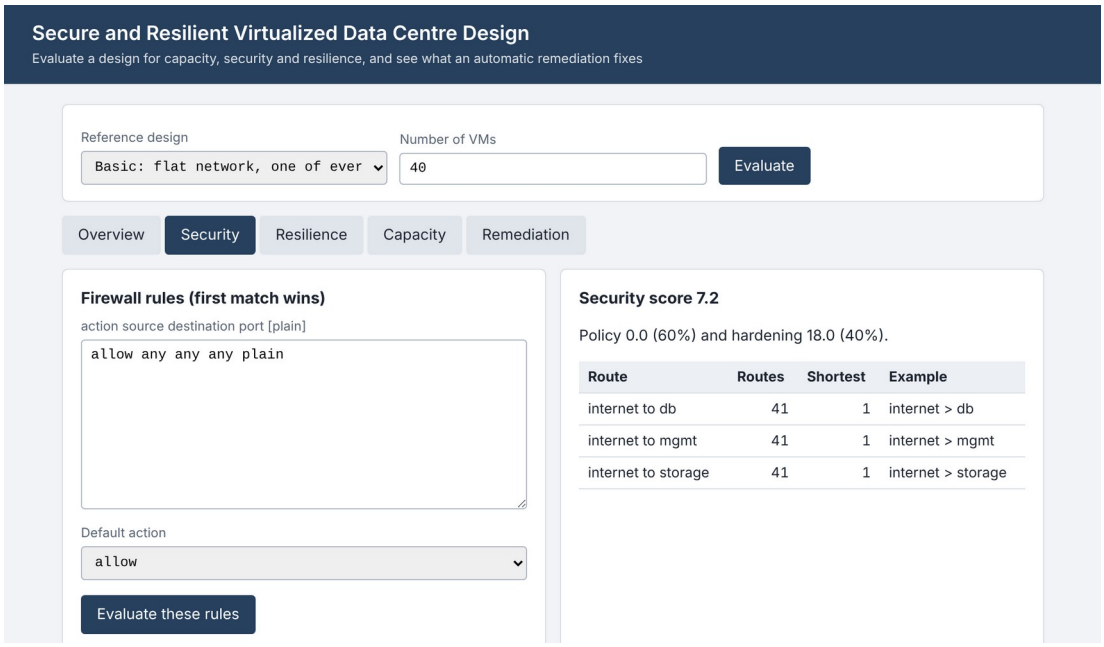


Figure 6.7: Security tab for the basic design: editable rules, findings, routes and checklist

Figure 6.7 shows the basic design: the single allow-all rule is in the editable list, the two critical findings are listed with their fixes, and the table of routes shows 41 routes to each critical zone, the shortest of one hop. Editing the rules and pressing the button re-evaluates the design at once, which turns the page into an exercise in writing a firewall policy.

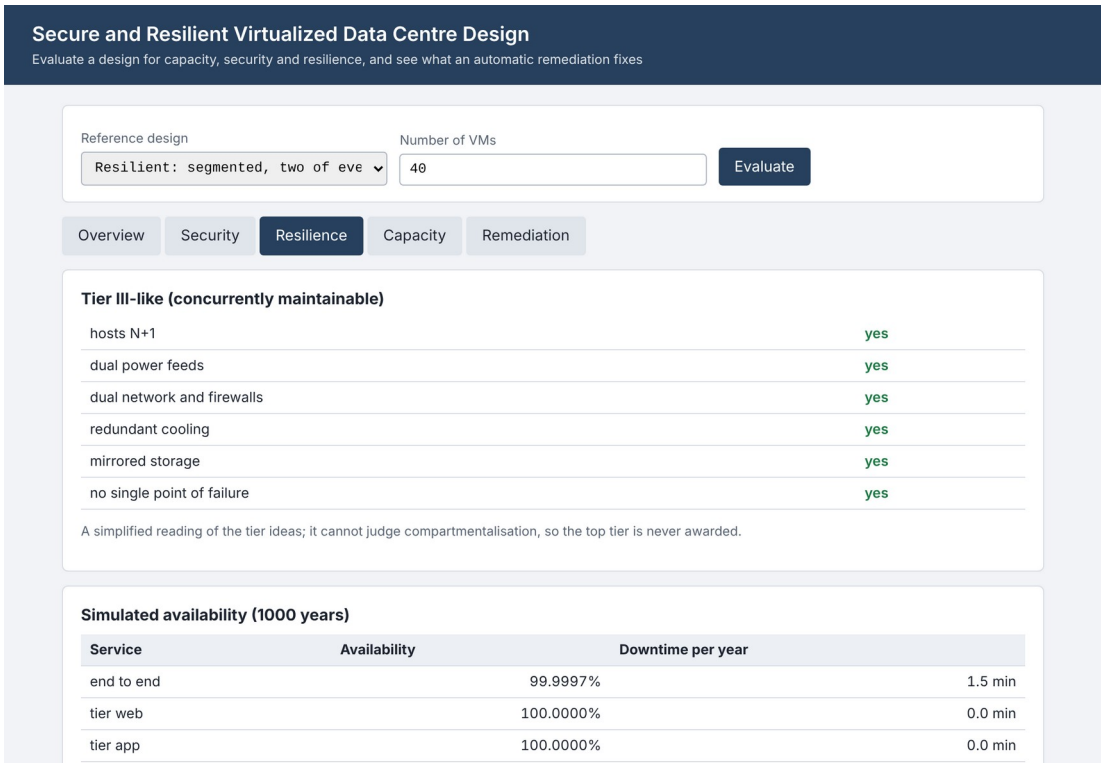


Figure 6.8: Resilience tab for the resilient design: tier reading, availability and the scenarios

Figure 6.8 shows the tier reading, in which all six properties are met, the simulated availability of each tier and the 19 single-failure scenarios. Each scenario gives the hosts

that go down, the VMs affected, restarted and still down, and the result. The two rack scenarios are the only ones that survive with a degradation: 13 VMs remain down after the loss of rack R1, but no service stops.

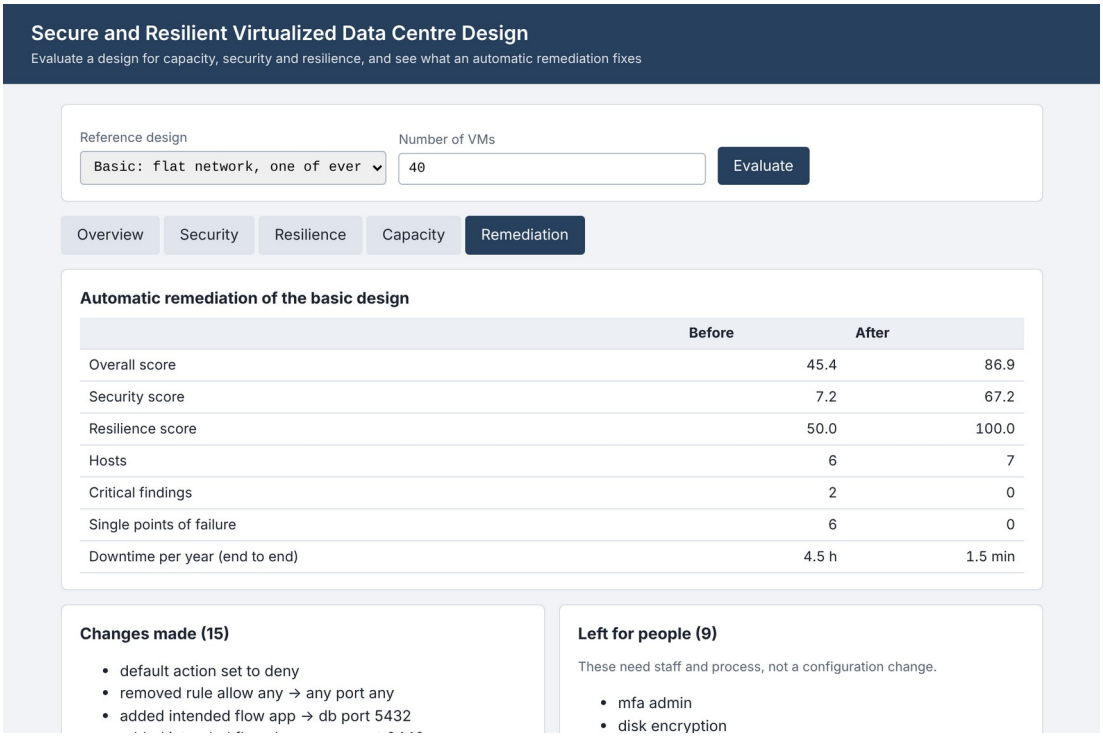


Figure 6.9: Remediation tab: before and after, changes made and what is left for people

Figure 6.9 shows the remediation of the basic design: the table of before and after, the 15 changes and the nine items left for people.

6.10 Discussion

The results support the aims of the project. The evaluator separates the three concerns and puts them on the same scale, so that a design review can say that the flat network is the problem of the basic design, that segmentation fixes the security problem but not the availability one, and that the single points of failure are fixed by doubling the paths and adding one host. The auto-remediation shows the practical limit of tooling: a program can repair a rule set and a topology, but the patching, authentication and backup items of the checklist are work for people, and they carry 40% of the security score.

The limits of the evidence must be stated as clearly. The designs are generated by the authors, the failure parameters are assumptions, the scoring weights are judgements, and the intended policy is the one of a simple three-tier service. The tool finds the weaknesses it was built to find: it does not find unknown vulnerabilities, misconfigurations inside a

VM, or faults in the physical separation of the paths. Its recommendations are therefore a first filter before a review by an engineer, not a substitute for it.

7. CONCLUSION

This mini project built a tool that evaluates the design of a virtualized data centre for capacity, security and resilience, and that repairs the part of a weak design that can be repaired by a program. A design is described as hosts, racks, power feeds, switches, cooling units, firewalls, storage arrays, VMs and ordered firewall rules. The tool places the VMs under replica constraints and tests N+1, evaluates the firewall with first-match semantics against an intended policy and searches for routes from the internet to the critical zones, fails every component and every pair of components, simulates the availability of each service including the effect of shared components, and reads the design in terms of the tier classification. 34 automated tests pass.

The experiments gave clear results. The basic flat design scored 45 out of 100, with an open route from the internet to the database and six single points of failure, and was down about 4.6 hours a year. Segmentation raised the security score to 82 and changed nothing in the availability. Doubling every path, mirroring the storage, using two racks and one more host gave a design with no single point of failure and about 3 minutes of downtime a year, which survives 95% of the double failures. The automatic remediation brought the basic design to 87 and left nine process items for people. All 476 injected firewall mistakes were found.

7.1 Limitations

- The tool evaluates a model of a design, not a real data centre; it does not read hypervisor or firewall configurations.
- The failure parameters, the scoring weights and the intended policy are assumptions or judgements of the authors.
- Software vulnerabilities, denial of service, insider threats and human error are not modelled; the hardening checklist is a list of yes or no answers.
- The simulation does not restart VMs automatically and treats failures as independent except through the shared components of the design.
- The tier reading is a simplified one and never awards the top tier; the physical separation of the paths cannot be seen by the model.
- Only one service architecture (web, app, db, management) is covered, with six fixed zones.

7.2 Future scope

- Read real configurations: firewall exports, hypervisor settings and an inventory, so that a real data centre can be evaluated.
- Add more zones and services, user-defined intended policies, and micro-segmentation at the VM level.
- Add disaster recovery across sites, with recovery time and recovery point objectives, and a second site in the failure scenarios.
- Model software faults and common-cause failures, such as a faulty hypervisor update that affects every host at once.
- Add power and cooling capacity (kilowatts per rack) so that the loss of a feed or a cooling unit reduces the capacity available.
- Add cost to the evaluation, so that the tool can find the cheapest design that reaches a target score.
- Add automatic restart and live migration to the simulation, and the time they take.

8. REFERENCE

[1] L. A. Barroso, J. Clidaras and U. Hölzle, The Datacenter as a Computer: Designing Warehouse-Scale Machines, 3rd ed. San Rafael, CA: Morgan & Claypool, 2018.

[2] W. P. Turner IV, J. H. Seader, V. Renaud and K. G. Brill, "Tier Classifications Define Site Infrastructure Performance," Uptime Institute, White Paper, 2008.

[3] P. Barham et al., "Xen and the art of virtualization," in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), 2003, pp. 164–177.

[4] C. Clark et al., "Live migration of virtual machines," in Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.

[5] E. G. Coffman Jr., M. R. Garey and D. S. Johnson, "Approximation algorithms for bin packing: a survey," in Approximation Algorithms for NP-Hard Problems, D. S. Hochbaum, Ed. Boston, MA: PWS Publishing, 1996, pp. 46–93.

[6] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975.

[7] K. Scarfone, M. Souppaya and P. Hoffman, "Guide to Security for Full Virtualization Technologies," NIST Special Publication 800-125, National Institute of Standards and Technology, 2011.

[8] K. Scarfone and P. Hoffman, "Guidelines on Firewalls and Firewall Policy," NIST Special Publication 800-41 Revision 1, National Institute of Standards and Technology, 2009.

[9] E. Al-Shaer and H. Hamed, "Discovery of policy anomalies in distributed firewalls," in Proceedings of IEEE INFOCOM, 2004, pp. 2605–2616.

[10] O. Sheyner, J. Haines, S. Jha, R. Lippmann and J. M. Wing, "Automated generation and analysis of attack graphs," in Proceedings of the IEEE Symposium on Security and Privacy, 2002, pp. 273–284.

[11] K. S. Trivedi, Probability and Statistics with Reliability, Queuing, and Computer Science Applications, 2nd ed. New York, NY: Wiley, 2002.

[12] R. Chandramouli, "Secure Virtual Network Configuration for Virtual Machine (VM) Protection," NIST Special Publication 800-125B, National Institute of Standards and Technology, 2016.

[13] J. Gray, "Why do computers stop and what can be done about it?" Tandem Computers, Technical Report 85.7, 1985.